



INFORME DEL TP

1ACC0218- TEORÍA DE COMPILADORES

Carrera de Ciencias de la Computación

Sección: 4889

Alumnos:	
Código	Nombres y apellidos
U202216662	Pamela Rivera Contreras
U202419995	Piero Antonio Aguilar Anticona
U20231E777	Leicy Cristell Cahuana Lopez

Docente: Jose Luis Soncco Alvarez

Tabla de Contenidos

1. Contexto.....	3
1.1. Problemática y Motivación.....	3
1.2. Objetivos.....	3
1.2.1. Objetivo General.....	3
1.2.2. Objetivos Específicos.....	4
2. Construcciones del Lenguaje VizFlow.....	4

1. Contexto

1.1. Problemática y Motivación

Actualmente, carreras como Economía, Estadística y Administración trabajan constantemente con datos para obtener información, realizar cálculos y generar gráficos. Sin embargo, no todos los usuarios tienen conocimientos de programación, por lo que realizar estas tareas mediante herramientas como Python puede resultar complicado. Para un análisis sencillo, normalmente es necesario utilizar diferentes librerías para cargar y organizar los datos, realizar operaciones sobre ellos y generar las visualizaciones.

Además, aunque existen herramientas visuales que facilitan el análisis de datos, estas pueden resultar poco prácticas cuando se busca realizar un proceso mediante un conjunto de instrucciones que pueda repetirse o modificarse fácilmente.

Por este motivo, proponemos VizFlow, un lenguaje diseñado específicamente para realizar análisis y visualización de datos de manera sencilla. La idea es que el usuario pueda indicar directamente las operaciones que desea realizar sobre un conjunto de datos, como cargar información, filtrar registros, realizar transformaciones, obtener estadísticas, agrupar datos y generar gráficos, sin tener que escribir código adicional para cada una de estas tareas.

Para facilitar este proceso, VizFlow utilizará un operador de tubería (`|>`), que permitirá encadenar las diferentes operaciones de forma secuencial sobre un mismo conjunto de datos. De esta manera, cada operación podrá tomar como entrada el resultado de la anterior, permitiendo expresar todo el proceso de análisis como un flujo continuo y evitando la necesidad de crear variables o instrucciones intermedias para cada paso.

El resultado final será un dashboard que reunirá las estadísticas y visualizaciones obtenidas a partir del análisis realizado. De esta manera, VizFlow busca simplificar el proceso de análisis y permitir que el usuario se concentre en interpretar sus datos.

1.2. Objetivos

1.2.1. Objetivo General

Diseñar e implementar VizFlow, un lenguaje orientado al análisis y visualización de datos que permita procesar información de manera sencilla y generar un dashboard a partir de las operaciones realizadas.

1.2.2. Objetivos Específicos

- Implementar el análisis léxico y sintáctico mediante ANTLR4 para reconocer las diferentes construcciones y operadores definidos en VizFlow.
- Desarrollar el análisis semántico para verificar la correcta utilización de los datos, como la existencia de las columnas utilizadas y la compatibilidad de tipos en las operaciones.
- Permitir la lectura e ingesta de datos tabulares en formato CSV.
- Implementar operaciones de manipulación de datos, como el filtrado mediante FILTER y la creación de nuevos valores mediante TRANSFORM.
- Incorporar operaciones de análisis estadístico como SUM, AVG, MIN, MAX y COUNT, junto con la posibilidad de agrupar los datos mediante GROUP_BY.
- Permitir la generación de visualizaciones mediante gráficos de barras, líneas y dispersión (BAR, LINE y SCATTER).
- Generar un dashboard como resultado final, integrando las estadísticas y visualizaciones obtenidas a partir del procesamiento de los datos.

2. Construcciones del Lenguaje VizFlow

Para organizar el lenguaje, se han definido cinco construcciones principales que representan las operaciones más importantes que el usuario podrá realizar sobre sus datos.

2.1. Sentencia de Carga de Datos (LOAD)

Esta construcción permite al usuario cargar un archivo de datos para comenzar el análisis.

Ejemplo 1: DATASET ventas = LOAD "ventas.csv";

Ejemplo 2: DATASET clientes = LOAD "clientes_2026.csv";

Ejemplo 3: DATASET notas = LOAD "evaluaciones.csv";

Ejemplo 4: DATASET inventario = LOAD "stock_almacen.csv";

Ejemplo 5: DATASET llamadas = LOAD "call_center_log.csv";

2.2. Sentencia de Filtrado (FILTER)

Permite seleccionar únicamente aquellas filas del dataset que cumplan con una condición lógica.

Ejemplo 1: ventas |> FILTER(monto > 500);

Ejemplo 2: clientes |> FILTER(edad >= 18);

Ejemplo 3: notas |> FILTER(promedio < 11);

Ejemplo 4: inventario |> FILTER(stock == 0);

Ejemplo 5: llamadas |> FILTER(duracion > 120);

2.3. Sentencia de Transformación Aritmética (TRANSFORM)

Permite calcular y agregar una nueva columna al dataset aplicando expresiones aritméticas sobre columnas existentes.

Ejemplo 1: ventas |> TRANSFORM(monto_igv = monto * 1.18);

Ejemplo 2: clientes |> TRANSFORM(edad_meses = edad * 12);

Ejemplo 3: notas |> TRANSFORM(nota_final = examen1 + examen2);

Ejemplo 4: inventario |> TRANSFORM(valor_total = precio * cantidad);

Ejemplo 5: ventas |> TRANSFORM(ganancia = precio_venta - costo_base);

2.4. Sentencia de Agrupamiento (GROUP_BY)

Esta construcción permite obtener información resumida mediante operaciones como SUM, AVG, MIN, MAX y COUNT, además de agrupar los resultados según una determinada columna.

Ejemplo 1: ventas |> GROUP_BY(categoria, SUM(monto));

Ejemplo 2: ventas |> GROUP_BY(region, AVG(monto));

Ejemplo 3: productos |> GROUP_BY(categoria, MIN(precio));

Ejemplo 4: productos |> GROUP_BY(categoria, MAX(precio));

Ejemplo 5: clientes |> GROUP_BY(region, COUNT());

2.5. Sentencia de Visualización (PLOT)

Permite representar los resultados del análisis mediante diferentes tipos de gráficos.

Ejemplo 1: ventas |> PLOT BAR(x=categoria, y=monto);

Ejemplo 2: ventas |> PLOT LINE(x=mes, y=monto);

Ejemplo 3: clientes |> PLOT BAR(x=region, y=COUNT());

Ejemplo 4: productos |> PLOT SCATTER(x=precio, y=ventas);

Ejemplo 5: alumnos |> PLOT BAR(x=curso, y=promedio);

3. Resultado final de Vizflow

Como resultado de la ejecución de las operaciones anteriores, VizFlow generará un dashboard que reunirá los resultados del análisis y las visualizaciones generadas a partir del dataset. De esta manera, el usuario podrá visualizar en un mismo espacio las estadísticas obtenidas y los gráficos correspondientes.

Para ejecutar el flujo completo, el lenguaje contará con la instrucción:

```
|> DASHBOARD
```

Esta instrucción indica que se deben ejecutar las operaciones definidas anteriormente y presentar sus resultados en un dashboard.

Ejemplo:

ventas

```
|> FILTER(monto > 500)
```

```
|> GROUP_BY(categoria, SUM(monto))
```

```
|> PLOT BAR(x=categoria, y=monto)
```

```
|> DASHBOARD;
```

En este caso, DASHBOARD ejecuta el flujo definido previamente y genera como resultado final un dashboard que contiene el resultado del agrupamiento y el gráfico correspondiente.

4. -